

 MULTI-AGENT

7 Multi-Agent Architectures in Real Products

One agent is powerful.
The right combination is unstoppable.

 Supervisor

 Planner-Executor

 Research Team

 Debate



AI Coach John

FOLLOW



 Repost

When Single-Agent Systems **Hit Their Limit**

✖ SINGLE AGENT LIMITS

- Fixed context window
- Sequential only — no parallelism
- Can't specialize per task
- Harder to debug
- One failure = entire task fails

✔ MULTI-AGENT SOLVES

- Multiple context windows
- Parallel execution
- Specialized per role
- Independent review
- Isolated failure handling

→ WHEN TO GO MULTI-AGENT

- Task requires more than 30 steps (single agent loses track)
- Different parts need different expertise (research vs writing vs coding)
- Speed is critical — parallel execution needed
- Quality must be independently verified



Rule: Don't go multi-agent until a single agent demonstrably fails on your task.



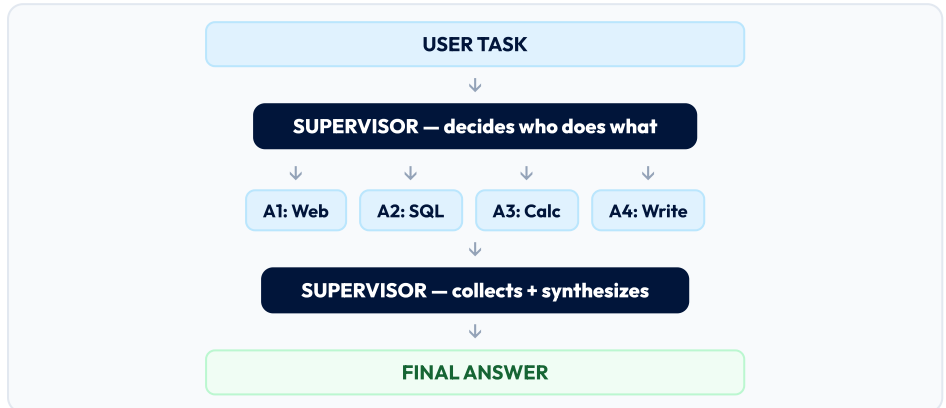
AI Coach John

FOLLOW



Repost

Supervisor Pattern — One Agent Directs Many



✓ BEST FOR

Research + reporting, customer support routing, data analysis pipelines, multi-step content creation.

</> FRAMEWORKS

LangGraph, CrewAI, AutoGen. Supervisor uses tool calling to select next worker agent.



Planner-Executor — Think First, Then Act

PLANNER AGENT

Claude Opus · no tools · just thinking



Step 1: search competitor X
Step 2: pull our pricing DB
Step 3: compare & draft report



EXECUTOR AGENT(S)

Claude Haiku · all tools · follow steps

★ WHY SEPARATE PLANNING FROM EXECUTION

- Planner can use expensive model without slowing throughput
- Executors use fast/cheap models — just follow steps
- Plan can be reviewed by human before execution begins
- Executor failure doesn't corrupt the plan — just retry the step



Best for: Data engineering, code generation + execution, document processing pipelines, automated testing.



AI Coach John

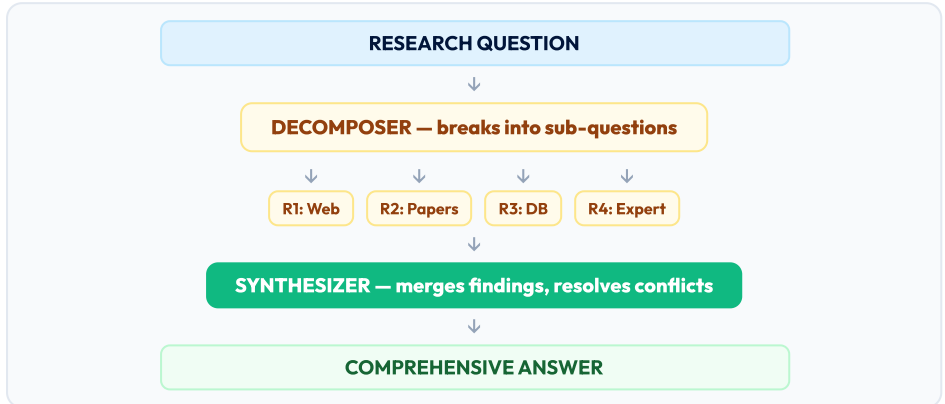
FOLLOW



PROTIBRIDGE
Smart Pro Safety Systems

 **Repost**

Research Team Pattern — Parallel Deep Research



✓ BEST FOR

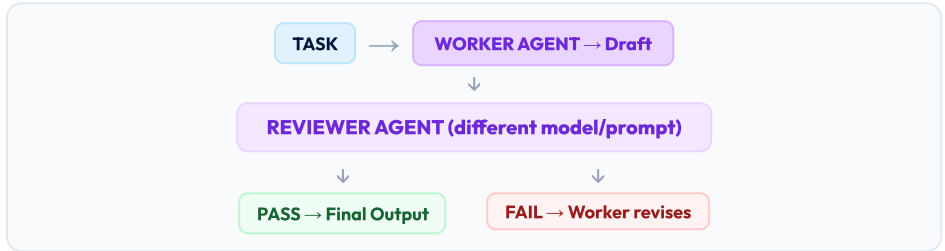
Due diligence, competitive intel, medical literature review, investment research.

💡 PRO TIP

Add a **CONFLICT RESOLVER** agent when researchers find contradicting information.



Reviewer Pattern — Independent Quality Control



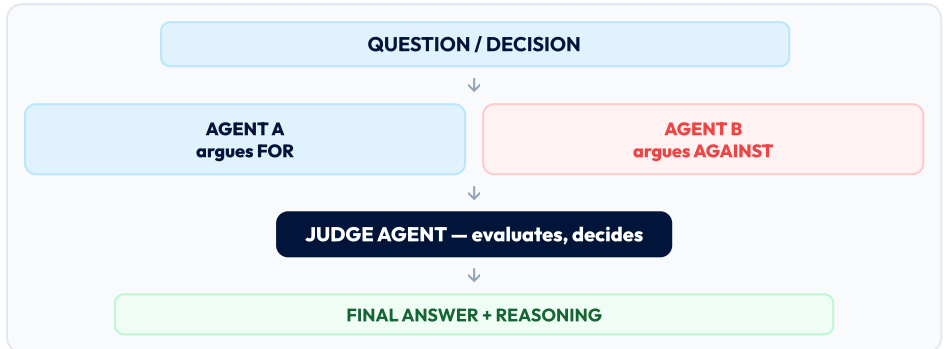
```

def review_loop(task: str, max_rounds: int = 3) → str:
    draft = worker_agent.generate(task)
    for round in range(max_rounds):
        review = reviewer_agent.evaluate(
            task=task, output=draft,
            criteria=["correctness", "completeness",
                    "security", "style"]
        )
        if review["verdict"] == "APPROVED":
            return draft
        draft = worker_agent.revise(
            draft=draft, feedback=review["feedback"]
        )
    return draft
  
```

★ **Key insight:** Same agent reviewing its own work = same blind spots. Independence is mandatory.



Debate Pattern — Find Truth Through Disagreement



REAL SCENARIO

Medical AI: Agent A proposes Diagnosis X. Agent B finds evidence contradicting X. Judge outputs: **"Most likely X (78%) but consider Y (22%). Reasoning: ..."**



Best for: High-stakes decisions, fact-checking, risk analysis, investment/strategy calls.



AI Coach John

FOLLOW



 **Repost**

Hierarchical Agents — Command Structure at Scale



☰ DESIGN PRINCIPLES

- Each level communicates only with level above/below
- Workers report *status*, not results — managers synthesize
- Executive controls budget and resources
- Any agent can escalate to their manager

⚠ **Limitation:** Communication overhead grows with depth. Max 3 levels for most use cases.



AI Coach John

FOLLOW



🔄 Repost

Swarm Agents — Emergent Behavior, No Control

NO CENTRAL CONTROLLER

A1

A2

A3

A4

A5

A6

Each agent: same base prompt · peer-to-peer comms · local decisions → emergent consensus

✓ BEST FOR

Brainstorming, simulations, creative exploration, adversarial red-teaming, consensus building.

✗ AVOID WHEN

Deterministic tasks required. Behavior is harder to predict and debug.



Framework: OpenAI Swarm (experimental), custom peer-to-peer messaging systems.



AI Coach John

FOLLOW



PROITBRIDGE
Direct For Smarter Futures

🔄 Repost

Which Pattern for Which Problem?

PATTERN	COORDINATION	PARALLELISM	QUALITY	COMPLEXITY
Supervisor	Central	High	Medium	Medium
Planner-Exec	Staged	High	Low-Med	Low
Research Team	Parallel	Very High	Medium	Medium
Reviewer	Sequential	Low	Very High	Low
Debate	Adversarial	Medium	Very High	Medium
Hierarchical	Tree	High	High	Very High
Swarm	Peer-to-peer	Very High	Low	Very High



AI Coach John

FOLLOW



 Repost

How Agents Talk to Each Other

 Message Passing Shared State Event-Driven Stream-based

```
from typing import TypedDict
from langgraph.graph import StateGraph

class AgentState(TypedDict):
    task: str
    research_results: list
    draft: str
    review_feedback: str
    final_output: str

# Each agent reads relevant fields, writes output
def research_agent(state: AgentState) → AgentState:
    results = do_research(state["task"])
    return {"research_results": results}

def writer_agent(state: AgentState) → AgentState:
    draft = write_content(
        state["task"],
        state["research_results"]
    )
    return {"draft": draft}

# Build graph
graph = StateGraph(AgentState)
graph.add_node("research", research_agent)
graph.add_node("write", writer_agent)
```

★ **Shared state (LangGraph)** is the most common pattern in production — flexible, debuggable, testable.



AI Coach John

FOLLOW



 Repost

What Nobody Tells You About **Multi-Agent** in **Production**

\$ Cost explosion — 7 agents × 5 tool calls = 35 LLM calls per request. Add cost guardrails with per-request budget caps.

🕒 Latency budget — With parallelism, you wait for the SLOWEST agent. Set per-agent timeouts or the whole system stalls.

🐛 Debugging — One wrong output propagates through 7 agents. You need per-agent tracing (LangSmith / LangFuse).

∞ Loop detection — Agents can get into infinite correction loops. max_iterations limits are non-negotiable.

```
class CostGuard:
    def __init__(self, max_cost_usd: float = 1.0):
        self.max_cost = max_cost_usd; self.spent = 0.0
    def check(self, estimated_cost: float) → bool:
        if self.spent + estimated_cost > self.max_cost:
            raise BudgetExceeded(f"Spent: ${self.spent:.2f}")
        self.spent += estimated_cost; return True
```



Multi-Agent Patterns Cheatsheet

PATTERN	USE WHEN	AVOID WHEN
Supervisor	Mixed subtasks need coordination	Simple linear tasks
Planner-Executor	Human review needed mid-task	Real-time requirements
Research Team	Parallel deep research needed	Single-source tasks
Reviewer	Output quality is critical	Speed > quality
Debate	Decision confidence must be justified	Clear-cut factual tasks
Hierarchical	Large-scale complex automation	Small / simple tasks
Swarm	Exploration, brainstorming	Deterministic tasks

💡 The Rule

Start with the simplest pattern that solves your problem.

A Reviewer Pattern catches most quality issues at 10% of the complexity of a full Debate Pattern.

